



CHASKIPC • HARDWARE & GAMING STORE

Sistema Distribuido para la Venta y Facturación de Componentes de Alta Gama

■ INTEGRANTE 1: ELICEO PARILLO MOSTAJO

- Rol: Arquitectura Backend, Infraestructura Spring Cloud & Persistencia.
- Microservicio Transaccional: pc-orden-ms (Cabecera-Detalle, IGV 18%, PostgreSQL :5433).
- Microservicio No Transaccional: pc-catalogo-ms (Catálogo de Hardware y Stock :8081).
- Componentes Core: Config Server (:8888), Eureka Server (:8761), API Gateway (:18080).
- Observabilidad: Prometheus (:19090) y Grafana (:13000) con dashboards JVM.

■ INTEGRANTE 2: LAURA VARGAS CRISTHIAN PAUL

- Rol: Pasarela de Pagos, Seguridad Perimetral y Contratos REST.
- Microservicio Transaccional: pc-pago-ms (Integración Mercado Pago Sandbox y Webhooks).
- Microservicio No Transaccional: pc-auth-ms (Gestión de Identidad y Control de Acceso).
- Seguridad Distribuida: Aislamiento perimetral en Gateway (cero acceso directo a microservicios).
- Configuración Segura: Variables de entorno para credenciales en perfil PROD.



LÍMITES DEL MONOLITO EN CHASKIPC

Justificación técnica del desacoplamiento hacia una arquitectura distribuida

■ CUELLO DE BOTELLA MONOLÍTICO

- Puntos Críticos Detectados:
 - • Caída total de la tienda ante picos de concurrencia en lanzamientos gamer.
 - • Bloqueo en BD compartida por bloqueo mutuo entre catálogo y checkout.
 - • Escalamiento ineficiente: obligaba a duplicar todo el artefacto monolítico pesado.
- Impacto al Negocio:
 - • Pérdida de órdenes en caliente y degradación de la experiencia de compra.

■ TRANSICIÓN A MICROSERVICIOS

- Desacoplamiento Estructural:
 - • Bases de datos independientes por servicio (Database-per-Service).
 - • Escalamiento elástico horizontal de réplicas transaccionales (:8082, :8083).
 - • Despliegues independientes sin interrumpir módulos de autenticación o inventario.
- Tolerancia a Fallos:
 - • Una falla en pagos o catálogo no detiene la navegación de clientes.

■ OBJETIVOS DE LA SOLUCIÓN

- Metas Arquitecturales S05:
 - 1. Descubrimiento dinámico de réplicas mediante Eureka Service Registry.
 - 2. Enrutamiento unificado y balanceo Round-Robin con Spring Cloud Gateway.
 - 3. Configuración externalizada versionada con Spring Cloud Config Server.
 - 4. Cálculo automático de IGV 18% y persistencia transaccional con abecedario-detalle.



ARQUITECTURA DISTRIBUIDA EN 4 CAPAS

Ecosistema desacoplado conforme a la Guía Oficial de Evaluación S05

■ CAPA 1: GATEWAY

- Spring Cloud Gateway (:18080):
 - • Punto único de entrada perimetral.
 - • Enrutamiento lb:// balanceado.
 - • Filtros globales de trazabilidad.
 - • Control de CORS unificado.

■ CAPA 2: REGISTRY

- Eureka Server (:8761):
 - • Service Registry en memoria.
 - • Heartbeats cada 30 segundos.
 - • Health check automático.
 - • Detección de réplicas en caliente.

■ CAPA 3: CONFIG

- Config Server (:8888):
 - • Repositorio local config-repo.
 - • Perfiles DEV vs PROD aislados.
 - • Refresh de beans en caliente.
 - • Centralización de application.yml.

■ CAPA 4: SERVICIOS

- Ecosistema de Microservicios:
 - • pc-orden-ms (:8082, :8083)
 - • pc-catalogo-ms (:8081)
 - • pc-pago-ms (:8084)
 - • pc-auth-ms (:8085)
 - • PostgreSQL (:5433)



FICHAS TÉCNICAS & CONTRATOS REST (2 POR INTEGRANTE)

Asignación equitativa de microservicios transaccionales y no transaccionales

■ ELICEO PARILLO: CATÁLOGO Y ÓRDENES

■ pc-orden-ms (Transaccional Core):

- • Cabecera-detalle con cálculo de Subtotal, IGV 18% y Total general.
- • POST /api/v1/ordenes (201 Created)
- • GET /api/v1/ordenes/{id} (200 OK / 404 Not Found)
- • GET /api/v1/ordenes/instancia (Comprueba balanceo Round-Robin)

■ pc-catalogo-ms (No Transaccional):

- • Gestión de productos (CPUs, GPUs, RAM, Motherboards) y stock.
- • GET /api/v1/productos (200 OK)
- • GET /api/v1/productos/{id} (200 OK / 404 Not Found)

■ CRISTHIAN PAUL: PAGOS Y AUTENTICACIÓN

■ pc-pago-ms (Transaccional):

- • Integración con SDK Mercado Pago Sandbox para cobros en línea.
- • Procesamiento de tarjeta, montos y webhooks de confirmación.

■ • POST /api/v1/pagos (201 Created / 400 Bad Request)

■ • POST /api/v1/pagos/webhook (200 OK)

■ pc-auth-ms (No Transaccional):

- • Gestión de identidad, roles y perfiles de cliente (DNI/RUC).
- • Seguridad perimetral validada desde el Gateway.
- • POST /api/v1/auth/login



CONFIGURACIÓN EXTERNALIZADA DEV VS PROD (FACTOR III)

El mismo artefacto JAR opera en desarrollo y producción sin recompilar

■ ■ PERFIL DEV (ORDEN-MS-DEV.YML)

- Base de datos local: localhost:5433/orden_db.
- DDL Hibernate: update para iteración ágil y pruebas de laboratorio.
- Nivel de Log: DEBUG para diagnóstico exhaustivo en terminal.
- Actuator: Expone health, info, metrics y prometheus.
- Eureka Server: http://localhost:8761/eureka/.

■ PERFIL PROD (ORDEN-MS-PROD.YML)

- Red interna Docker aislada: pagatu-prod-net.
- Credenciales protegidas: Variables \${POSTGRES_USER} y \${POSTGRES_PASSWORD}.
- DDL Hibernate: validate (máxima protección de integridad relational).
- Actuator seguro: Detalles de endpoints internos ocultos al exterior.
- Eureka Server: Resolución mediante nombre de contenedor en Docker.



MATRIZ INTEGRAL DE MAPEO MULTICAPA

Mapeo desacoplado en Gateway, Eureka, Hibernate JPA y Códigos HTTP

■ ■ MATRIZ DE MAPEO DEL SISTEMA DISTRIBUIDO CHASKIPC

- 1. Mapeo Gateway: `/api/v1/ordenes/**` -> `lb://pagatu-orden-ms` (Round Robin balanceado entre réplicas :8082 y :8083).
- 2. Mapeo Gateway: `/api/v1/productos/**` -> `lb://pagatu-catalogo-ms` (Puerto :8081).
- 3. Mapeo Gateway: `/api/v1/pagos/**` -> `lb://pc-pago-ms` | `/api/v1/auth/**` -> `lb://pc-auth-ms`.
- 4. Mapeo Eureka: `spring.application.name` coincide en mayúsculas; `instance-id` dinámico evita colisiones en el mismo host.
- 5. Mapeo JPA: `@Entity` Orden (`@Table 'ordenes'`) y `DetalleOrden` (`@Table 'orden_detalles'`) con relación `@OneToMany`.
- 6. Mapeo Jackson: `@JsonManagedReference` y `@JsonBackReference` previenen bucle cíclico en serialización JSON.
- 7. Mapeo Códigos HTTP: 201 Created (alta exitosa), 200 OK (consulta), 400 Bad Request (validación), 404 Not Found.



GUION DE SUSTENTACIÓN A DOS VOCES (18 MINUTOS)

Estructura oficial minuto a minuto basada en la Tabla 5 de la Guía S05

■■ BLOQUE 1: EXPOSICIÓN (8 MIN)

- Min 0 - 4 (Eliceo Parillo):
 - • Problemática del monolito.
 - • Arquitectura base 4 capas.
 - • Eureka, Config y Gateway.
- Min 4 - 8 (Cristhian Paul):
 - • Fichas técnicas y contratos REST.
 - • Modelo cabecera-detalle con IGV.
 - • Perfiles DEV vs PROD.

■ BLOQUE 2: DEMO EN VIVO (5 MIN)

- Min 8 - 10.5 (Cristhian Paul):
 - • Ejecución de Caso 201 Created.
 - • Verificación de IGV 18% en BD.
 - • Prueba de validación 400 y 404.
- Min 10.5 - 13 (Eliceo Parillo):
 - • 4 peticiones de balanceo Round-Robin.
 - • Tolerancia a fallos apagando 8082.

■■ BLOQUE 3: PREGUNTAS (5 MIN)

- Min 13 - 18 (Ronda Individual):
 - • Eliceo responde P1 y P2 del balotario.
 - • Cristhian Paul responde P3, P4 y P5.
 - • Calificación esperada: 20 / 20 Puntos.

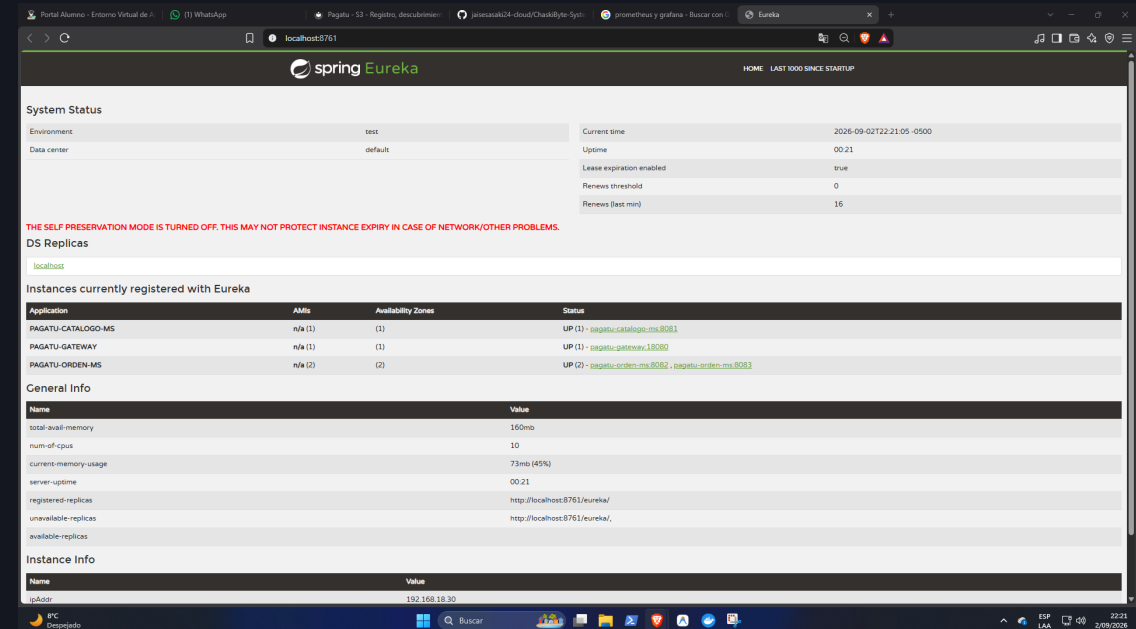


EJEMPLOS DE FUNCIONAMIENTO REAL DEL ECOSISTEMA

Capturas operativas del registro en Eureka Server y ejecución HTTP en consola

■ 7 EVIDENCIAS, URLS Y COMANDOS

- 01. Config (:8888): <http://localhost:8888/pagatu-orden-ms/dev>
 - CMD: `cd pagatu-config ; .\mvnw.cmd spring-boot:run`
- 02. Eureka (:8761): <http://localhost:8761>
 - CMD: `cd pagatu-eureka ; .\mvnw.cmd spring-boot:run`
- 03. Catálogo (:8081): <http://localhost:18080/api/v1/productos>
 - CMD: `cd pagatu-catalogo-ms ; .\mvnw.cmd spring-boot:run`
- 04. Orden R1 (:8082): <http://localhost:8082/api/v1/ordenes/instancia>
 - CMD: `cd pagatu-orden-ms ; .\mvnw.cmd spring-boot:run`
- 05. Orden R2 (:8083): <http://localhost:8083/api/v1/ordenes/instancia>
 - CMD: `$env:SERVER_PORT='8083' ; .\mvnw.cmd spring-boot:run`
- 06. Dashboard (:8761): <http://localhost:8761>
 - PowerShell: `Start-Process http://localhost:8761`
- 07. Pruebas CRUD (:18080): <http://localhost:18080/api/v1/ordenes>
 - PowerShell: `Invoke-RestMethod -Uri ../ordenes -Method POST`



■ ENLACES DE APERTURA DIRECTA DE EVIDENCIAS:

- Captura Alta Resolución: [evidencia_6_eureka_web_dashboard.png](#)
- Eureka Web: <http://localhost:8761> | Cockpit Web: <http://localhost:5050/presentacion.html>



CENTRO DE MANDO EN VIVO & PRUEBAS OFICIALES

Control de scripts, monitor de salud en tiempo real y ejecución de pruebas con 1 clic

■ LANZADORES E INICIADORES DE CARPETA

- [■] Iniciar Todo el Ecosistema: Arranca los 5 microservicios en consolas separadas.
- [■] Iniciar PostgreSQL: Levanta contenedor en puerto 5433.
- [■] Iniciar Observabilidad: Arranca Prometheus y Grafana.
- [■] Limpiar Puertos: Libera puertos bloqueados en la PC de laboratorio.
- Enlaces Rápidos:
 - • Eureka: <http://localhost:8761>
 - • Gateway Órdenes: <http://localhost:18080/api/v1/ordenes>
 - • Config Server: <http://localhost:8888/pagatu-orden-ms/dev>

■ PRUEBAS OFICIALES ASIGNADAS POR INTEGRANTE

- [Cristhian Paul] ■ Caso 201 Created:
 - • Crea orden de compra con IGV 18% y muestra ID generado.
- [Cristhian Paul] ■ Caso 400 Bad Request:
 - • Envía payload vacío y valida captura de error sin caída.
- [Cristhian Paul] ■ Caso 404 Not Found:
 - • Consulta ID inexistente 99999 y comprueba código 404.
- [Eliceo Parillo] ■ Balanceo Round-Robin:
 - • 4 peticiones demostrando alternancia entre 8082 y 8083.
- [Eliceo Parillo] ■ Tolerancia a Fallos en Caliente:
 - • Detiene réplica 8082 y el Gateway responde desde 8083.